# Practical Numerical Simulations HW3

Eoghan Stafford

November 2025

## 1 Laplace Equation With Dirichlet Boundary Conditions

The Laplace Equation in 2-D is given by,

$$\nabla^2\phi(x,y) = \frac{\partial^2\phi}{\partial x^2} + \frac{\partial^2\phi}{\partial y^2} = 0.$$

The field $\phi(x,y)$ satisfies the Laplace equation in the unit square $[1,0]\times[0,1]$. We aim to numerically compute $\phi(x,y)$ within this region under certain boundary conditions using the Successive Over-Relaxation (SOR) Method.

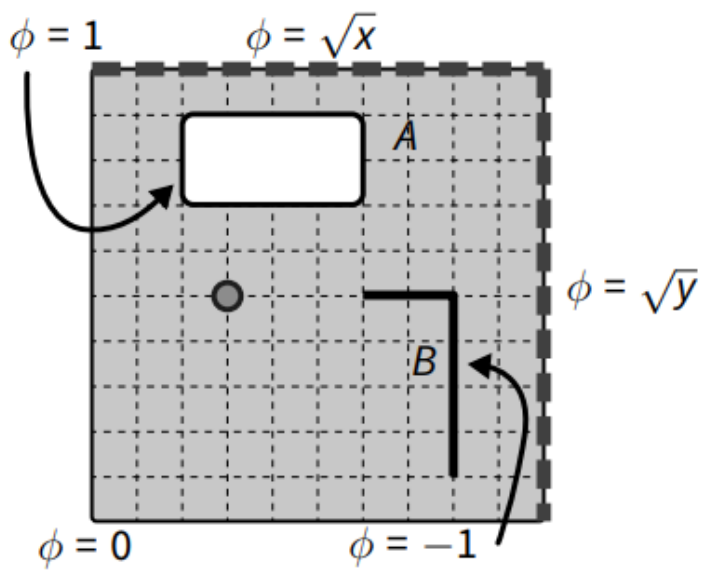


Figure 1: Region we are solving for $\phi(x,y)$ on

First we must discretize the Laplace equation using Taylor series. For a function $f(x+h)$ with $h << 1$,

$$f(x+h) = \sum_{n=0}^{\infty} \frac{f^n(x)}{n!} h^n$$

$$f(x+h) = f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + O(h^4)$$

$$f(x-h) = f(x) - f'(x)h + \frac{f''(x)}{2!}h^2 - \frac{f'''(x)}{3!}h^3 + O(h^4)$$

$$f(x+h) + f(x-h) = 2f(x) + f''(x)h^2 + O(h^4)$$

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

Using this for our Laplace equation:

$$\frac{\partial^2 \phi}{\partial x^2} = \frac{\phi(x+h,y) - 2\phi(x,y) + \phi(x-h,y)}{h^2}$$

$$\frac{\partial^2 \phi}{\partial y^2} = \frac{\phi(x,y+h) - 2\phi(x,y) + \phi(x,y-h)}{h^2}$$

$$\nabla^2 \phi(x,y) = \frac{\phi(x+h,y) - 2\phi(x,y) + \phi(x-h,y) + \phi(x,y+h) - 2\phi(x,y) + \phi(x,y-h)}{h^2} = 0$$

$$4\phi(x,y) = \phi(x+h,y) + \phi(x-h,y) + \phi(x,y+h) + \phi(x,y-h)$$

If we take $h$ to be our step size and $\phi(x,y)$ to be the value of $\phi$ at indices i and j corresponding to some point $(x,y)$ within our region, we get the discretized Laplace equation to be,

$$\phi_{i,j} = \frac{1}{4}(\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1})$$

We now construct a grid on the unit square to represent the Laplace equation at each point. The domain is divided into N points along the x and y directions with spacing $h_x = h_y = \frac{1}{N}$. Only the interior grid points where there are no Dirichlet boundary conditions contain unknown values of $\phi(x,y)$.

For clarity, we consider the case of a $3 \times 3$ interior grid.

$$\begin{matrix} \phi_{i-1,j-1} & \phi_{i-1,j} & \phi_{i-1,j+1} \\ \phi_{i,j-1} & \phi_{i,j} & \phi_{i,j+1} \\ \phi_{i+1,j-1} & \phi_{i+1,j} & \phi_{i+1,j+1} \end{matrix}$$

At each point on this grid, the discretized Laplace equation is satisfied along with the boundary conditions. Defining the Laplace equation column by column and declaring points outside the grid are boundary terms we get a system of equations:

$$4\phi_{i-1,j-1} - \phi_{i,j-1} - \phi_{i-1,j} = b_{i-1,j-1}$$

$$4\phi_{i,j-1} - \phi_{i+1,j-1} - \phi_{i-1,j-1} - \phi_{i,j} = b_{i,j-1}$$

$$4\phi_{i+1,j-1} - \phi_{i,j-1} - \phi_{i+1,j} = b_{i+1,j-1}$$

$$\vdots$$

$$4\phi_{i,j} - \phi_{i+1,j} - \phi_{i-1,j} - \phi_{i,j+1} - \phi_{i,j-1} = 0$$

$$\vdots$$

$$4\phi_{i+1,j+1} - \phi_{i,j+1} - \phi_{i+1,j} = b_{i+1,j+1}$$

where each term $b_{i,j}$ contains the contributions of the boundary values adjacent to point $(i, j)$. We can write this in vector notation as:

$$A\vec{\Phi} = \vec{b}$$

$$\begin{bmatrix} 4 & -1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 4 & 0 & 0 & -1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 4 & -1 & 0 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 4 & -1 & 0 & -1 & 0 \\ 0 & 0 & -1 & 0 & -1 & 4 & 0 & 0 & -1 \\ 0 & 0 & 0 & -1 & 0 & 0 & 4 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & -1 & 4 \end{bmatrix} \begin{bmatrix} \phi_{i-1,j-1} \\ \phi_{i,j-1} \\ \phi_{i+1,j-1} \\ \phi_{i-1,j} \\ \phi_{i,j} \\ \phi_{i+1,j} \\ \phi_{i-1,j+1} \\ \phi_{i,j+1} \\ \phi_{i+1,j+1} \end{bmatrix} = \begin{bmatrix} b_{i-1,j-1} \\ b_{i,j-1} \\ b_{i+1,j-1} \\ b_{i-1,j} \\ b_{i,j} \\ b_{i+1,j} \\ b_{i-1,j+1} \\ b_{i,j+1} \\ b_{i+1,j+1} \end{bmatrix}$$

This matrix, A, generalizes to to any size $N \times N$ interior grid. The finite-difference approximation we computed above for the Laplace equation gives the same five-point approximation which only depends on a points nearest neighbors to the left, right, top and bottom of it.

In order to apply the SOR method, we must flatten this 2-D grid into a 1-D array. If we flatten the grid column by column using $p = i + jN$, we create a one to one mapping from the indices $(i, j)$ to $p$. For example, a grid of the following,

$$\begin{matrix} \phi_{0,0} & \phi_{0,1} & \phi_{0,2} \\ \phi_{1,0} & \phi_{1,1} & \phi_{1,2} \\ \phi_{2,0} & \phi_{2,1} & \phi_{2,2} \end{matrix}$$

Is flattened column by column using

$$p = i + jN, \qquad i, j = 0, 1, 2$$

$$\Phi = \begin{bmatrix} \phi_{0,0} \\ \phi_{1,0} \\ \phi_{2,0} \\ \phi_{0,1} \\ \phi_{1,1} \\ \phi_{2,1} \\ \phi_{0,2} \\ \phi_{1,2} \\ \phi_{2,2} \end{bmatrix} \rightarrow \begin{bmatrix} \phi_0 \\ \phi_1 \\ \phi_2 \\ \phi_3 \\ \phi_4 \\ \phi_5 \\ \phi_6 \\ \phi_7 \\ \phi_8 \end{bmatrix}$$

We now have the complete linear system $A\vec{\Phi} = \vec{b}$ which represents the Laplace equation at all interior points of the unit square with the Dirichlet boundary conditions accounted for in $\vec{b}$.

The SOR method is an iterative technique used to solve large systems of linear equations, $A\vec{x} = \vec{b}$, especially those that arise from discretized partial differential equations. It computes each component of $\vec{x}$ at iteration $k+1$ by using the most recently updated values of the earlier components

and the values from iteration $k$ for the components that have not yet been updated. It is defined as,

$$x_i^{k+1} = (1-\omega)x_i^k + \frac{\omega}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{k+1} - \sum_{j>i} a_{ij}x_j^k\right)$$

Applying this to our system with our index $p$,

$$\Phi_p^{k+1} = (1-\omega)\Phi_p^k + \frac{\omega}{a_{pp}}\left(b_p - \sum_{q<p} a_{pq}\Phi_q^{k+1} - \sum_{q>p} a_{pq}\Phi_q^k\right)$$

The only non-zero matrix elements $a_{pq}$ for a pure interior point occur at $q = p-1, p-N, p+1, p+N$, where N is the number of grid points along each axis. Since we are flattening column by column, the indices $p-1$ and $p+1$ correspond to points above and below the point we are updating respectively. The indices $p-N$ and $p+N$ correspond to points which are to the left and right of the point we are updating. The diagonal elements $a_{pp} = 4$ and the off-diagonal non-zero elements are all $a_{pq} = -1$. For a pure interior point, $b_p = 0$. Our SOR update method for any pure interior point in the grid is now,

$$\Phi_p^{k+1} = (1-\omega)\Phi_p^k + \frac{\omega}{4}\left(\Phi_{p-1}^{k+1} + \Phi_{p-N}^{k+1} + \Phi_{p+1}^k + \Phi_{p+N}^k\right)$$

We can now implement this iterative SOR method into C++ to compute the value of $\phi(x, y)$ within the unit square subject to Dirichlet boundary conditions.

## 1.1 Results

A good choice for the relaxation parameter $\omega$ was found by computing,

$$\Delta = \max_{i,j}\left|\phi_{i,j}^k - \phi_{i,j}^{k-1}\right| < 1 \times 10^{-6}$$

for every interior grid point $(i, j)$ for $1.0 \leq \omega \leq 2.0$. Once $\Delta$ was below the specified tolerance of $1 \times 10^{-6}$ the number of iterations it took for the corresponding $\omega$ was recorded. Using a $100 \times 100$ grid, the fastest convergence was achieved using $\omega = 1.90$ which required 210 iterations to fall below the specified tolerance. This process was completed for a range of grid sizes.

| **Grid Size $N \times N$** | **Optimal $\omega$** | **Iterations taken to converge** | **Final Change in $\phi_{i,j}^k$** |
|---|---|---|---|
| $100 \times 100$ | 1.90 | 210 | $9.59945 \times 10^{-7}$ |
| $110 \times 110$ | 1.91 | 233 | $9.15154 \times 10^{-7}$ |
| $120 \times 120$ | 1.91 | 249 | $9.97698 \times 10^{-7}$ |
| $130 \times 130$ | 1.92 | 269 | $9.49412 \times 10^{-7}$ |
| $140 \times 140$ | 1.92 | 296 | $9.78653 \times 10^{-7}$ |
| $150 \times 150$ | 1.93 | 309 | $9.81087 \times 10^{-7}$ |
| $160 \times 160$ | 1.93 | 330 | $9.95565 \times 10^{-7}$ |
| $170 \times 170$ | 1.94 | 356 | $9.92413 \times 10^{-7}$ |
| $180 \times 180$ | 1.94 | 368 | $9.28078 \times 10^{-7}$ |
| $190 \times 190$ | 1.94 | 389 | $9.86542 \times 10^{-7}$ |
| $200 \times 200$ | 1.95 | 428 | $9.17755 \times 10^{-7}$ |

Table 1: Optimal $\omega$ for different grid sizes under Dirichlet boundary conditions

To compute the value of $\frac{\partial \phi}{\partial y}(0.3, 0.5)$ we use a similar method to what was used to discretize the Laplace equation.

$$\phi(x, y+h) = \phi(x, y) + \frac{\partial \phi}{\partial y} h + \frac{\partial^2 \phi}{\partial y^2} \frac{h^2}{2!} + O(h^3)$$

$$\phi(x, y-h) = \phi(x, y) - \frac{\partial \phi}{\partial y} h + \frac{\partial^2 \phi}{\partial y^2} \frac{h^2}{2!} + O(h^3)$$

$$\phi(x, y+h) - \phi(x, y-h) = 2 \frac{\partial \phi}{\partial y} h$$

$$\frac{\partial \phi(x, y)}{\partial y} = \frac{\phi(x, y+h) - \phi(x, y-h)}{2h}$$

Which can be discretized as,

$$\frac{\partial \phi(x_i, y_j)}{\partial y} = \frac{\phi_{i,j+1} - \phi_{i,j-1}}{2h_y}$$

This was implemented in the executable part of our program after the maximum change in $\phi(x, y)$ over all interior points was less than the specified tolerance of $1 \times 10^{-6}$. This ensures that the value of the field $\phi(x, y)$ has fully converged for the specific grid size it is being evaluated on.

| **Grid Size** $N \times N$ | $\frac{\partial \phi(0.3,0.5)}{\partial y}$ |
|---|---|
| $100 \times 100$ | 2.406248 |
| $110 \times 110$ | 2.405240 |
| $120 \times 120$ | 2.404343 |
| $130 \times 130$ | 2.403676 |
| $140 \times 140$ | 2.403029 |
| $150 \times 150$ | 2.402564 |
| $160 \times 160$ | 2.402058 |
| $170 \times 170$ | 2.401754 |
| $180 \times 180$ | 2.401364 |
| $190 \times 190$ | 2.400995 |
| $200 \times 200$ | 2.400841 |
| $300 \times 300$ | 2.398927 |
| $500 \times 500$ | 2.397108 |

Table 2: $\frac{\partial \phi(0.3,0.5)}{\partial y}$ evaluated on different grid sizes for Dirichlet boundary conditions

As the grid size increases, the numerical approximation for $\frac{\partial \phi(0.3,0.5)}{\partial y}$ becomes more accurate. This behavior is expected since refining the grid reduces the difference between two successive points meaning the computed value $\frac{\partial \phi(x_i,y_j)}{\partial y}$ using the formula derived above will also reduce with an increasing grid size. At the same time, a finer grid requires more SOR iterations to reach the convergence criterion

$$\max_{i,j} \left| \phi_{i,j}^k - \phi_{i,j}^{k-1} \right| < 1 \times 10^{-6}$$

since the number of unknown values of $\phi(x_i, y_j)$ grows in accordance with $N^2$. For example, a $300 \times 300$ grid requires 604 iterations to converge with $\omega_{opt} = 1.96$.

To assess the accuracy of $\frac{\partial\phi(0.3,0.5)}{\partial y}$, we compare its value across different grid sizes. The results for grids of size $200 \times 200$ and $300 \times 300$ differ by,

$$\left|2.400841 - 2.398927\right| = 1.914 \times 10^{-3} < 5 \times 10^{-3}$$

which is less than the threshold required for two values to agree within 3 s.f., $5 \times 10^{-3}$. Thus, the final value accurate to 3 s.f. is,

$$\frac{\partial\phi(0.3, 0.5)}{\partial y} = 2.40$$

# 2 Laplace Equation With Neumann Boundary Conditions

## 2.1 Results

The optimal value of the relaxation parameter $\omega$ was computed for the following grid sizes with Neumann boundary conditions on the left and lower boundaries of the grid,

| **Grid Size** $N \times N$ | **Optimal** $\omega$ | **Iterations taken to converge** | **Final Change in** $\phi_{i,j}^k$ |
|---|---|---|---|
| $100 \times 100$ | 1.97 | 434 | $9.68568 \times 10^{-7}$ |
| $200 \times 200$ | 1.98 | 788 | $9.95164 \times 10^{-7}$ |
| $300 \times 300$ | 1.99 | 1221 | $9.87193 \times 10^{-7}$ |

Table 3: Optimal $\omega$ for different grid sizes under Neumann boundary conditions

The same method was used for calculating $\frac{\partial\phi(0.3,0.5)}{\partial y}$ under Neumann boundary conditions as the previous section. We run the program until $\phi(x, y)$ has converged at every point for a range of grid sizes.

| **Grid Size** $N \times N$ | $\frac{\partial\phi(0.3,0.5)}{\partial y}$ |
|---|---|
| $100 \times 100$ | 2.496002 |
| $200 \times 200$ | 2.485412 |
| $300 \times 300$ | 2.482007 |
| $400 \times 400$ | 2.480140 |

Table 4: $\frac{\partial\phi(0.3,0.5)}{\partial y}$ evaluated on different grid sizes for Neumann boundary conditions

Again, comparing the value of $\frac{\partial\phi(0.3,0.5)}{\partial y}$ for grid sizes of $300 \times 300$ and $400 \times 400$, noting that the more we increase the grid size the more we reduce the finite–difference error and therefore find a more accurate approximation of the field, we find:

$$\left|2.482007 - 2.480140\right| = 1.867 \times 10^{-3} < 5 \times 10^{-3}$$

Thus, the final value accurate to 3 s.f. under Neumann boundary conditions is,

$$\frac{\partial\phi(0.3, 0.5)}{\partial y} = 2.48$$

# 3 Plots

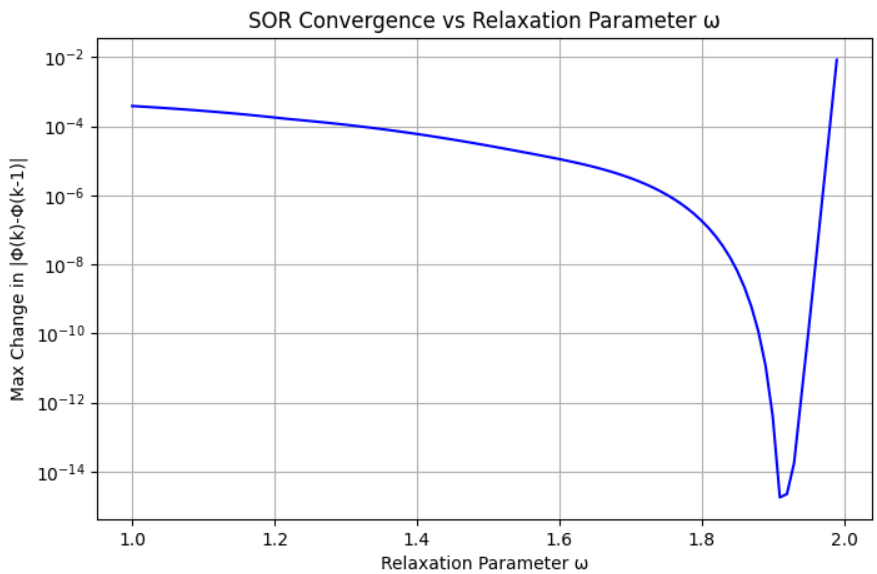


Figure 2: SOR Convergence vs Relaxation Parameter $\omega$ for Dirichlet boundary conditions

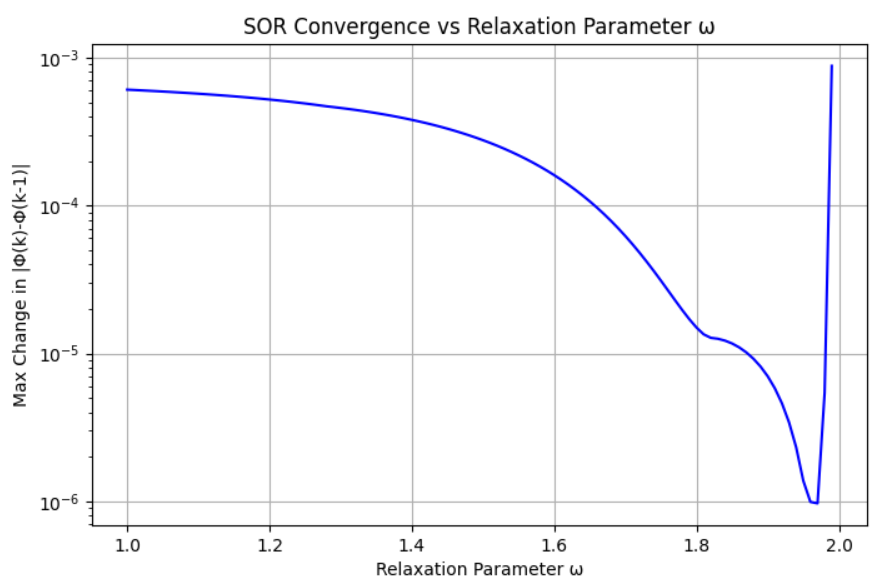


Figure 3: SOR Convergence vs Relaxation Parameter $\omega$ for Neumann boundary conditions

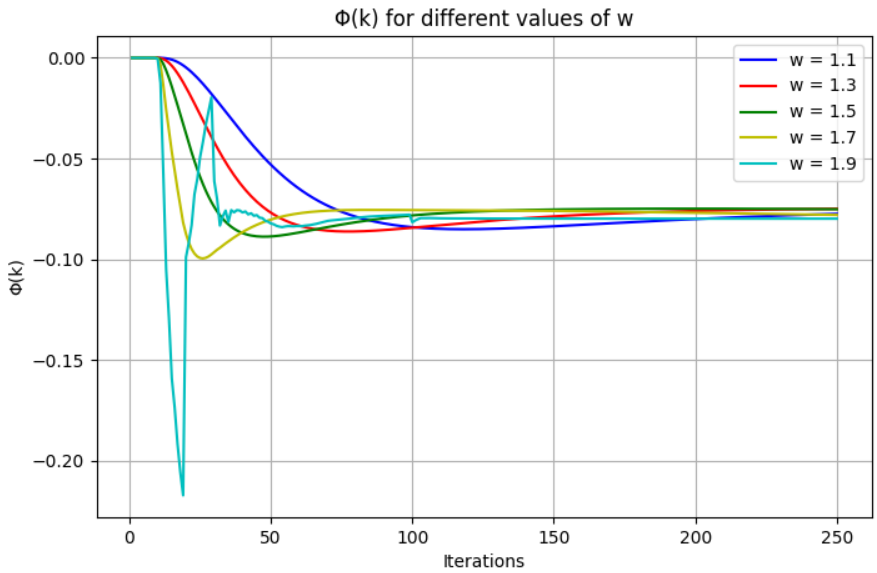


Figure 4: $\phi(x, y)$ at center point of grid for different values of $\omega$ under Dirichlet boundary conditions